RAPPORT DE PROJET

Module Drupal 10 Personnalisé

SCHEDULEHUB

Système de Prise de Rendez-vous

Nom complet : Ayoub Barahal

Technologie : Drupal 10 / PHP 8.2 / MySQL

Module : appointment

Statut : Terminé

"Réservation, administration et notifications email en un seul module."

Table des matières

1 Introduction **3**
- 1.1 Contexte du projet . . . 3
- 1.2 Fonctionnalités principales . . . 3
- 1.3 Technologies utilisées . . . 3

2 Architecture du module **4**
- 2.1 Structure des fichiers . . . 4
- 2.2 Concepts Drupal utilisés . . . 5

3 Fichiers racine du module **6**
- 3.1 appointment.info.yml — Carte d'identité . . . 6
- 3.2 appointment.routing.yml — Table des URLs . . . 6
  - 3.2.1 Routes publiques . . . 6
  - 3.2.2 Routes d'administration . . . 7
- 3.3 appointment.services.yml — Registre des services . . . 7
- 3.4 appointment.permissions.yml — Droits d'accès . . . 7
- 3.5 appointment.libraries.yml — Ressources frontend . . . 7
- 3.6 appointment.links.menu.yml et links.task.yml . . . 8
- 3.7 config/install/appointment.settings.yml — Configuration par défaut . . . 8

4 Hooks Drupal (appointment.module) **9**
- 4.1 hook_theme() — Enregistrement des templates . . . 9
- 4.2 hook_page_attachments() — Chargement des assets . . . 9
- 4.3 hook_mail() — Construction des emails . . . 9
- 4.4 hook_cron() — Rappels automatiques . . . 10

5 Installation et mises à jour (appointment.install) **11**

6 Entités (src/Entity/) **12**
- 6.1 Pourquoi des entités personnalisées ? . . . 12
- 6.2 AgencyEntity — Entité Agence . . . 12
- 6.3 AppointmentEntity — Entité Rendez-vous . . . 12

7 Services (src/Service/) **14**
- 7.1 Pourquoi des services ? . . . 14
- 7.2 AppointmentManagerService — Le cerveau du module . . . 14
  - 7.2.1 Détail : vérification de disponibilité . . . 14
- 7.3 EmailService — Le facteur du module . . . 15

8 Formulaires (src/Form/) **16**
- 8.1 AppointmentBookForm — Wizard 6 étapes . . . 16
  - 8.1.1 Les 6 étapes . . . 16
  - 8.1.2 Méthodes . . . 16
- 8.2 AppointmentCancelForm — Annulation . . . 17
- 8.3 AppointmentSettingsForm — Configuration . . . 17
- 8.4 AppointmentExportForm — Export CSV . . . 17
- 8.5 AppointmentLookupForm — Recherche . . . 17

9 Contrôleurs (src/Controller/) **18**

10.1.1 appointment-confirmation.html.twig 19
10.1.2 appointment-list.html.twig 19
10.2 Templates d'emails 19

11 Plugin QueueWorker **20**
11.1 AppointmentEmailQueueWorker 20

12 Système d'emails — Flux complet **21**
12.1 Envoi immédiat (confirmation, modification, annulation) 21
12.2 Envoi différé (rappels via cron) 21
12.3 Pourquoi une file d'attente ? 22
12.4 Test avec MailHog 22

13 Justification des choix techniques **23**

14 Commandes Drush utiles **24**

15 Dépannage **24**

1 Introduction

1.1 Contexte du projet

ScheduleHub est un module Drupal 10 personnalisé développé dans le cadre d'un projet de fin de formation. Il permet aux visiteurs (anonymes ou connectés) de **réserver, modifier, annuler et consulter** des rendez-vous via un formulaire wizard en 6 étapes, avec une interface d'administration complète et un système de notifications par email.

1.2 Fonctionnalités principales

- Formulaire de réservation en 6 étapes avec navigation avant/arrière
- Calendrier interactif (FullCalendar) pour la sélection de créneaux
- Modification et annulation de rendez-vous existants
- Recherche de rendez-vous par email ou téléphone
- Tableau de bord administrateur avec statistiques
- Liste filtrée et paginée des rendez-vous (admin)
- Export CSV par lot (Batch API)
- 4 types d'emails HTML : confirmation, modification, annulation, rappel
- Emails de rappel automatiques via Queue + Cron
- Configuration modulaire (durée des créneaux, horaires, rappels)
- 4 permissions granulaires

1.3 Technologies utilisées

Technologie	Rôle
Drupal 10	CMS / Framework PHP
PHP 8.2	Langage serveur
MySQL	Base de données
Twig	Moteur de templates
FullCalendar 6.x	Calendrier JavaScript (CDN)
Docker + MailHog	Test des emails en développement
Drush	CLI pour Drupal

2 Architecture du module

2.1 Structure des fichiers

```
appointment/
+-- appointment.info.yml
+-- appointment.install
+-- appointment.module
+-- appointment.routing.yml
+-- appointment.services.yml
+-- appointment.permissions.yml
+-- appointment.libraries.yml
+-- appointment.links.menu.yml
+-- appointment.links.task.yml
+-- config/install/appointment.settings.yml
+-- css/appointment.admin.css
+-- js/appointment.dialog_stack.js
+-- templates/
|   +-- appointment-confirmation.html.twig
|   +-- appointment-list.html.twig
|   +-- email/
|       +-- appointment-email-confirmation.html.twig
|       +-- appointment-email-modification.html.twig
|       +-- appointment-email-cancellation.html.twig
|       +-- appointment-email-reminder.html.twig
+-- src/
    +-- Controller/
    |   +-- AppointmentController.php
    |   +-- AppointmentAdminController.php
    +-- Entity/
    |   +-- AgencyEntity.php
    |   +-- AppointmentEntity.php
    +-- Form/
    |   +-- AppointmentBookForm.php
    |   +-- AppointmentCancelForm.php
    |   +-- AppointmentLookupForm.php
    |   +-- AppointmentSettingsForm.php
    |   +-- AppointmentExportForm.php
    +-- Service/
    |   +-- AppointmentManagerService.php
    |   +-- EmailService.php
    +-- Plugin/QueueWorker/
        +-- AppointmentEmailQueueWorker.php
```

Listing 1 – Arborescence du module appointment

2.2 Concepts Drupal utilisés

Concept	Fichier(s)	Rôle
Module info	`appointment.info.yml`	Déclare le module auprès de Drupal
Routing	`appointment.routing.yml`	Associe chaque URL à un contrôleur ou formulaire
Entity	`src/Entity/*.php`	Type de données personnalisé avec table SQL dédiée
Form	`src/Form/*.php`	Construit, valide et traite les formulaires HTML
Controller	`src/Controller/*.php`	Retourne des pages (HTML ou JSON)
Service	`src/Service/*.php`	Logique métier réutilisable, injectable
QueueWorker	`src/Plugin/QueueWorker/`	Traite les tâches en file d'attente (cron)
Config	`config/install/*.yml`	Valeurs de configuration par défaut
Permissions	`appointment.permissions.yml`	Droits d'accès personnalisés
TempStore	`AppointmentBookForm.php`	Stockage temporaire entre les étapes du wizard
Batch API	`AppointmentExportForm.php`	Export CSV par lots

3 Fichiers racine du module

3.1 appointment.info.yml — Carte d'identité

Ce fichier est **obligatoire** pour tout module Drupal. Sans lui, Drupal ne reconnaît pas le module.

```yaml
name: Appointment Booking System
type: module
description: 'Appointment booking for anonymous and authenticated users.'
package: Custom
core_version_requirement: ^10
dependencies:
  - drupal:user
  - drupal:options
  - drupal:taxonomy
  - drupal:datetime
```

Clé	Rôle
`name`	Nom affiché dans l'interface admin
`type: module`	Indique à Drupal que c'est un module (pas un thème)
`core_version_requirement`	Compatible Drupal 10 uniquement
`dependencies`	Modules core requis — si un manque, Drupal refuse l'activation

3.2 appointment.routing.yml — Table des URLs

Ce fichier associe chaque URL à une classe PHP. Il définit **14 routes** au total.

3.2.1 Routes publiques

URL	Description
`/appointment/book`	Démarrer le wizard (étape 1)
`/appointment/book/{step}`	Aller à une étape spécifique (1–6)
`/appointment/{id}/edit`	Modifier un rendez-vous
`/appointment/{id}/edit/{step}`	Modifier à une étape spécifique
`/appointment/{id}/cancel`	Annuler un rendez-vous
`/appointment/{id}/confirmation`	Page de confirmation
`/appointment/my`	Mes rendez-vous
`/appointment/lookup`	Rechercher par email/téléphone
`/api/appointment/booked-slots`	API JSON pour le calendrier

3.2.2 Routes d'administration

URL	Description
`/admin/structure/appointment`	Tableau de bord
`/admin/.../list`	Liste filtrée et paginée
`/admin/.../export`	Export CSV
`/admin/.../csv`	Téléchargement du CSV
`/admin/.../settings`	Configuration du module

3.3 appointment.services.yml — Registre des services

Déclare les 2 services du module auprès du **conteneur de dépendances** de Drupal. Grâce à ce fichier, n'importe quelle classe peut demander un service sans le créer manuellement — c'est le principe de l'**injection de dépendances**.

```yaml
services:
  appointment.manager:
    class: Drupal\appointment\Service\AppointmentManagerService
    arguments:
      ["@entity_type.manager", "@date.formatter", "@datetime.time"]

  appointment.email_service:
    class: Drupal\appointment\Service\EmailService
    arguments:
      ["@plugin.manager.mail", "@logger.factory",
       "@language_manager", "@queue"]
```

Service	Arguments injectés	Pourquoi
`appointment.manager`	Entity manager, date formatter, time	Accès aux entités, formatage des dates, heure actuelle
`appointment.email_service`	Mail manager, logger, language, queue	Envoi de mails, logs, langue, file d'attente

3.4 appointment.permissions.yml — Droits d'accès

Permission	Description
`administer appointments`	Accès complet aux pages d'administration
`view own appointments`	Voir ses propres rendez-vous
`book appointments`	Créer un nouveau rendez-vous
`manage own appointments`	Modifier/annuler ses propres rendez-vous

3.5 appointment.libraries.yml — Ressources frontend

Déclare 3 bibliothèques CSS/JS :

Bibliothèque	Contenu
`appointment.frontend`	JS dialog-stack + dépendances jQuery, AJAX Drupal
`fullcalendar`	FullCalendar 6.x depuis CDN (calendrier étape 4)
`appointment.calendar`	JS personnalisé pour initialiser le calendrier

3.6 appointment.links.menu.yml et links.task.yml

- `links.menu.yml` : Ajoute le lien *Appointment Management* dans le menu admin sous *Structure*
- `links.task.yml` : Définit les 4 onglets admin : **Dashboard**, **All Appointments**, **Export CSV**, **Settings**

3.7 config/install/appointment.settings.yml — Configuration par défaut

```yaml
slot_duration: 60
working_hours_start: "08:00"
working_hours_end: "18:00"
max_advance_days: 90
csv_batch_size: 100
notification_email: ""
reminder_hours_before: 24
```

Paramètre	Défaut	Description
`slot_duration`	60 min	Durée d'un créneau
`working_hours_start`	08 :00	Début des heures ouvrables
`working_hours_end`	18 :00	Fin des heures ouvrables
`max_advance_days`	90	Jours max de réservation à l'avance
`csv_batch_size`	100	Lignes par lot lors de l'export CSV
`notification_email`	(vide)	Email admin (sinon email du site)
`reminder_hours_before`	24h	Heures avant le RDV pour le rappel

4 Hooks Drupal (appointment.module)

Le fichier `appointment.module` contient 4 hooks que Drupal appelle automatiquement.

4.1 hook_theme() — Enregistrement des templates

Enregistre **6 templates Twig** auprès de Drupal : 2 pages et 4 emails.

```
function appointment_theme(): array {
    return [
        'appointment_confirmation' => [
            'variables' => [
                'appointment' => NULL,
                'edit_url' => NULL,
                'cancel_url' => NULL,
            ],
            'template' => 'appointment-confirmation',
        ],
        'appointment_email_confirmation' => [
            'variables' => [
                'appointment' => NULL,
                'site_name' => NULL,
                'manage_url' => NULL,
                'date_label' => NULL,
            ],
            'template' => 'email/appointment-email-confirmation',
        ],
        // ... modification, cancellation, reminder
    ];
}
```

Sans ce hook, Drupal ne sait pas que les fichiers Twig existent.

4.2 hook_page_attachments() — Chargement des assets

Attache la bibliothèque JS/CSS `appointment.frontend` sur toutes les pages du site.

```
function appointment_page_attachments(array &$attachments): void {
    $attachments['#attached']['library'][]
        = 'appointment/appointment.frontend';
}
```

4.3 hook_mail() — Construction des emails

Reçoit le type d'email, rend le template Twig correspondant en HTML, et met le header `Content-Type: text/html`.

```
function appointment_mail(string $key, array &$message,
                          array $params): void {
```

```
    $theme_map = [
        'confirmation' => 'appointment_email_confirmation',
        'modification' => 'appointment_email_modification',
        'cancellation' => 'appointment_email_cancellation',
        'reminder'     => 'appointment_email_reminder',
    ];
    $build = ['#theme' => $theme_map[$key], ...];
    $html = $renderer->renderPlain($build);
    $message['body'][] = $html;
    $message['headers']['Content-Type'] = 'text/html; charset=UTF-8';
}
```

4.4 hook_cron() — Rappels automatiques

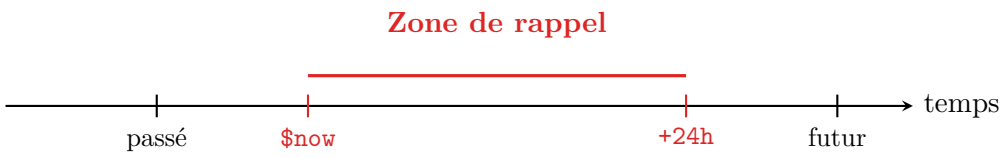
Cherche les rendez-vous à venir (dans les prochaines X heures) qui n'ont pas encore reçu de rappel, et les met en file d'attente.

```
function appointment_cron(): void {
    $hours_before = $config->get('reminder_hours_before'); // 24
    $now = \Drupal::time()->getRequestTime();
    $reminder_window = $now + ($hours_before * 3600);

    $ids = \Drupal::entityQuery('appointment')
        ->condition('status', ['booked', 'confirmed'], 'IN')
        ->condition('start_time', $now, '>')
        ->condition('start_time', $reminder_window, '<=')
        ->condition('reminder_sent', 0)
        ->execute();

    foreach ($appointments as $appointment) {
        $email_service->queueAppointmentEmail($id, 'reminder');
        $appointment->set('reminder_sent', TRUE);
        $appointment->save(); // Anti-doublon
    }
}
```

Visualisation de la fenêtre de rappel :



5 Installation et mises à jour (appointment.install)

Fonction	Rôle
`appointment_install()`	Crée le vocabulaire `appointment_type`, 3 termes, 2 agences de démo, 1 rendez-vous de test
`appointment_update_10001()`	Crée les tables `agency` et `appointment` si manquantes
`appointment_update_10002()`	Re-seed les données si les tables sont vides
`appointment_update_10003()`	Ajoute le statut `confirmed` + installe la config par défaut
`appointment_update_10004()`	Ajoute le champ `reminder_sent` + config `reminder_hours_before`

Les update hooks sont numérotés (10001, 10002...). Drupal les exécute dans l'ordre via `drush updb` et retient lesquels ont déjà tourné.

6 Entités (src/Entity/)

6.1 Pourquoi des entités personnalisées ?

Les nodes Drupal sont faits pour du contenu éditorial (articles, pages). Les données du module — agences et rendez-vous — sont des **données structurées** avec des champs précis. Les entités personnalisées offrent des **tables SQL dédiées**, plus performantes et plus propres.

6.2 AgencyEntity — Entité Agence

Propriété	Valeur
Fichier	`src/Entity/AgencyEntity.php`
Table SQL	`agency`
Hérite de	`ContentEntityBase`

Champs définis dans `baseFieldDefinitions()` :

Champ	Type	Description
`name`	string	Nom de l'agence
`address`	string_long	Adresse complète
`advisers`	string_long	Liste des conseillers en JSON
`status`	boolean	Actif / Inactif
`created`	created	Date de création
`changed`	changed	Date de modification

6.3 AppointmentEntity — Entité Rendez-vous

Propriété	Valeur
Fichier	`src/Entity/AppointmentEntity.php`
Table SQL	`appointment`
Hérite de	`ContentEntityBase`

15 champs définis dans `baseFieldDefinitions()` :

Champ	Type	Description
`uid`	entity_reference	Utilisateur Drupal
`agency`	entity_reference	Référence vers AgencyEntity
`adviser_name`	string	Nom du conseiller
`adviser_email`	string	Email du conseiller
`appointment_type`	entity_reference	Référence vers taxonomy
`start_time`	timestamp	Début du rendez-vous
`end_time`	timestamp	Fin du rendez-vous
`client_name`	string	Nom du client
`client_email`	email	Email du client
`client_phone`	string	Téléphone du client
`notes`	string_long	Notes optionnelles
`status`	list_string	booked / confirmed / cancelled
`access_token`	string	Token SHA-256 pour les anonymes
`reminder_sent`	boolean	Rappel déjà envoyé ?
`created / changed`	timestamp	Dates de suivi

7 Services (src/Service/)

7.1 Pourquoi des services ?

Le principe de **séparation des responsabilités** : le formulaire ne doit pas contenir la logique de vérification de disponibilité ou d'envoi d'email. Les services sont des classes réutilisables, déclarées dans `services.yml` et injectées via le conteneur de dépendances de Drupal.

7.2 AppointmentManagerService — Le cerveau du module

Propriété	Valeur
Service ID	`appointment.manager`
Arguments	`entity_type.manager`, `date.formatter`, `datetime.time`

Méthode	Rôle
`getAgencyOptions()`	Cherche les agences actives en base → retourne `[id => nom]` pour les dropdowns
`getAdviserOptions($id)`	Lit le JSON des conseillers d'une agence → retourne `[email => "Nom (email)"]`
`getAppointmentTypeOptions()`	Charge les termes de taxonomie → retourne `[tid => nom]`
`isSlotAvailable()`	Vérifie qu'aucun RDV actif ne chevauche le créneau demandé pour un conseiller donné
`saveAppointment()`	Crée ou met à jour une entité `appointment` à partir d'un tableau de valeurs
`getAppointments()`	Récupère les RDV d'un utilisateur (par uid) ou d'un anonyme (par email)
`formatTime()`	Convertit un timestamp UNIX en `Y-m-d H:i` lisible

7.2.1 Détail : vérification de disponibilité

```php
public function isSlotAvailable(string $adviser_email,
    int $start, int $end, ?int $exclude_id = NULL): bool
{
    // Charge tous les RDV "booked" du conseiller
    foreach ($appointments as $appointment) {
        $existing_start = $appointment->get('start_time')->value;
        $existing_end   = $appointment->get('end_time')->value;
        // Chevauchement = start < existing_end AND end > existing_start
        if ($start < $existing_end && $end > $existing_start) {
            return FALSE;
        }
```

```
    }
    return TRUE;
}
```

7.3 EmailService — Le facteur du module

Propriété	Valeur
Service ID	`appointment.email_service`
Arguments	`plugin.manager.mail`, `logger.factory`, `language_manager`, `queue`

Méthode	Rôle
`sendAppointmentEmail()`	Envoie **immédiatement** un email HTML via Drupal Mail API (SMTP → MailHog)
`queueAppointmentEmail()`	Met un email dans la **file d'attente** `appointment_email` pour traitement différé

```php
public function sendAppointmentEmail(
    AppointmentEntity $appointment, string $mail_key): void
{
    $to = $appointment->get('client_email')->value;
    $result = $this->mailManager->mail(
        'appointment', $mail_key, $to, $langcode, $params
    );
}

public function queueAppointmentEmail(
    int $appointment_id, string $mail_key): void
{
    $queue = $this->queueFactory->get('appointment_email');
    $queue->createItem([
        'appointment_id' => $appointment_id,
        'mail_key' => $mail_key,
    ]);
}
```

8 Formulaires (src/Form/)

8.1 AppointmentBookForm — Wizard 6 étapes

Propriété	Valeur
Hérite de	`FormBase`
Constante	`TOTAL_STEPS = 6`
Stockage inter-étapes	Private TempStore

8.1.1 Les 6 étapes

1. **Agence** — Choisir une agence dans un menu déroulant
2. **Type** — Choisir le type de rendez-vous (Consultation, Follow-up, Support)
3. **Conseiller** — Choisir un conseiller de l'agence sélectionnée
4. **Date & Heure** — Choisir un créneau via le calendrier FullCalendar
5. **Informations personnelles** — Nom, email, téléphone, notes
6. **Confirmation** — Récapitulatif complet, bouton confirmer

8.1.2 Méthodes

Méthode	Rôle
`buildForm()`	Construit les champs de l'étape courante
`validateForm()`	Vérifie étape 4 (créneau dispo ?) et étape 5 (téléphone valide ?)
`submitForm()`	Navigation avant/arrière via TempStore. Étape 6 : sauvegarde + email
`getCurrentStep()`	Lit l'URL pour savoir l'étape courante (1 à 6)
`getStoreKey()`	Génère la clé TempStore : `new_uid_session` ou `edit_id`
`stepUrl()`	Construit l'URL de l'étape suivante/précédente
`storeCurrentStepValues()`	Sauvegarde les valeurs courantes dans le TempStore
`buildSummaryMarkup()`	Génère le récapitulatif HTML pour l'étape 6
`seedDataFromAppointment()`	Pré-remplit le wizard en mode modification
`loadAgencyOptions()`	Charge la liste des agences
`loadTypeOptions()`	Charge la liste des types
`loadAdviserOptions()`	Charge la liste des conseillers d'une agence
`loadAdviserName()`	Trouve le nom du conseiller par son email
`isSlotAvailable()`	Vérifie la disponibilité d'un créneau

8.2 AppointmentCancelForm — Annulation

Hérite de `ConfirmFormBase`. Demande « Êtes-vous sûr ? », puis passe le statut à `cancelled` et envoie l'email d'annulation.

Méthode	Rôle
`getQuestion()`	Texte : « Voulez-vous vraiment annuler le RDV #5 ? »
`getCancelUrl()`	URL de retour si l'utilisateur dit Non
`submitForm()`	`status = cancelled` + email + redirect

8.3 AppointmentSettingsForm — Configuration

Hérite de `ConfigFormBase`. Formulaire admin pour régler les 7 paramètres du module.

Méthode	Rôle
`buildForm()`	Affiche les 7 champs de configuration
`validateForm()`	Vérifie le format `HH:MM` des horaires
`submitForm()`	Sauvegarde dans `appointment.settings`

8.4 AppointmentExportForm — Export CSV

Hérite de `FormBase`. Utilise le **Batch API** pour exporter sans timeout.

Méthode	Rôle
`buildForm()`	Affiche les filtres (statut, agence, dates)
`submitForm()`	Lance le batch : découpe les RDV en lots de 100
`processBatch()`	Traite un lot : écrit 100 lignes CSV
`batchFinished()`	Affiche le lien de téléchargement

8.5 AppointmentLookupForm — Recherche

Formulaire simple pour les visiteurs anonymes : choix radio (email ou téléphone) + champ texte. Redirige vers `/appointment/my?email=...`.

9 Contrôleurs (src/Controller/)

9.1 AppointmentController — Pages publiques

Méthode	Rôle
`confirmation()`	Page « Votre RDV est confirmé » avec détails + liens modifier/annuler
`myAppointments()`	Liste des RDV de l'utilisateur courant
`bookedSlots()`	API JSON pour FullCalendar : retourne les créneaux déjà pris
`loadTypeOptions()`	Charge les types de RDV pour le filtre

9.2 AppointmentAdminController — Back-office

Accessible uniquement avec la permission `administer appointments`.

Méthode	Rôle
`dashboard()`	Statistiques : total, réservés, confirmés, annulés, aujourd'hui
`listAppointments()`	Tableau filtrable et paginé de tous les RDV
`buildSelectOptions()`	Utilitaire HTML pour les filtres `<option>`
`downloadCsv()`	Sert le fichier CSV généré, puis le supprime du serveur

10 Templates Twig (templates/)

Twig est le moteur de templates de Drupal. Il sépare la logique (PHP) de la présentation (HTML).

10.1 Templates de pages

10.1.1 appointment-confirmation.html.twig

Page affichée après création d'un rendez-vous. Contient :

- Icône et titre « Votre rendez-vous est confirmé ! »
- Tableau de détails : référence, client, email, agence, conseiller, type, date, statut
- 3 boutons : Modifier / Annuler / Rechercher

Variables reçues du contrôleur : `appointment`, `edit_url`, `cancel_url`.

10.1.2 appointment-list.html.twig

Page « Mes rendez-vous ». Affiche :

- Si aucun RDV : message + bouton « Prendre un rendez-vous »
- Si RDV existants : tableau avec colonnes ID, Agence, Conseiller, Type, Date, Statut, Actions

Variables reçues : `rows`, `filters`, `lookup_url`.

10.2 Templates d'emails

Tous les templates email utilisent du **CSS inline** car les clients email ne supportent pas les feuilles CSS externes.

Template	Couleur	Contenu
`email-confirmation`	Bleu (`#0074bd`)	« Rendez-vous confirmé » + détails + bouton « Gérer »
`email-modification`	Jaune (`#ffc107`)	« Rendez-vous modifié » + nouveaux détails
`email-cancellation`	Rouge (`#dc3545`)	« Rendez-vous annulé » + date barrée + bouton re-réserver
`email-reminder`	Teal (`#17a2b8`)	« Rappel » + « Dans environ X heures » + détails

Variables communes : `appointment`, `site_name`, `manage_url`, `date_label`. Le template reminder reçoit aussi `hours_until`.

11 Plugin QueueWorker

11.1 AppointmentEmailQueueWorker

Propriété	Valeur
Fichier	`src/Plugin/QueueWorker/AppointmentEmailQueueWorker.php`
Hérite de	`QueueWorkerBase`
Implémente	`ContainerFactoryPluginInterface`
Queue ID	`appointment_email`
Temps cron	30 secondes max par exécution

Méthode principale : `processItem($data)`

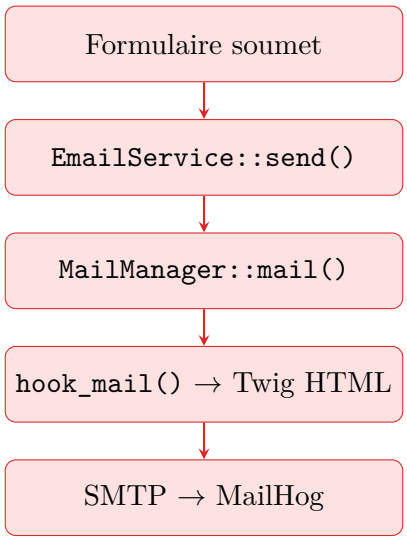
```
public function processItem($data): void {
    $appointment = $this->entityTypeManager
        ->getStorage('appointment')
        ->load($data['appointment_id']);

    $this->emailService->sendAppointmentEmail(
        $appointment,
        $data['mail_key'],
        $data['extra_params'] ?? []
    );
}
```

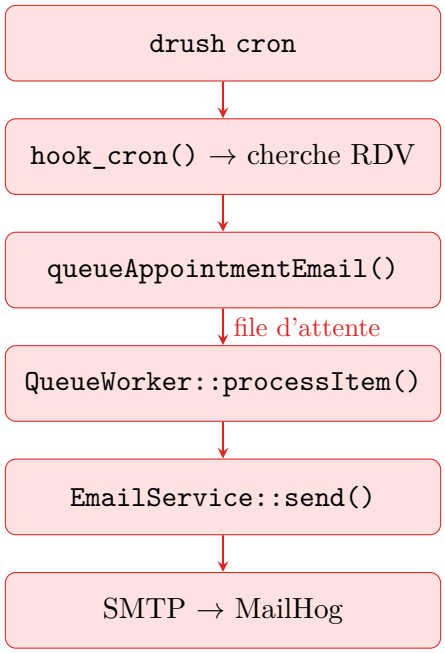
Reçoit un item de la queue → charge le RDV par ID → envoie l'email via `EmailService`.

12 Système d'emails — Flux complet

12.1 Envoi immédiat (confirmation, modification, annulation)



12.2 Envoi différé (rappels via cron)



12.3 Pourquoi une file d'attente ?

Envoi direct dans le cron	Via la queue
500 RDV = 500 emails d'un coup → timeout PHP	Traitement par lots, max 30s par cron
Si un email échoue → tout plante	Un échec → les autres continuent
Pas de retry	Drupal re-tente automatiquement

12.4 Test avec MailHog

```
# Lancer MailHog via Docker
docker run -d --name mailhog -p 1025:1025 -p 8025:8025 mailhog/mailhog

# Configurer Drupal SMTP -> localhost:1025
# Voir les emails captures : http://localhost:8025

# Tester les rappels
drush cron
```

13 Justification des choix techniques

Choix	Justification
Drupal 10	Fournit un système d'entités, routing, formulaires, permissions et cron prêt à l'emploi
Entités custom (pas des nodes)	Tables SQL dédiées, plus performantes pour des données structurées non-éditoriales
Wizard multi-étapes	Évite un formulaire trop long. TempStore côté serveur = plus sécurisé que le navigateur
Services injectables	Séparation des responsabilités. Logique métier testable et réutilisable
Queue pour les rappels	Traitement par lots (max 30s), retry automatique, pas de timeout
Batch API pour CSV	Découpe l'export en lots de 100 lignes avec barre de progression
MailHog	Simule un serveur SMTP en dev. Aucun email réel envoyé
CSS inline dans les emails	Les clients email (Gmail, Outlook) ne supportent pas les feuilles CSS externes
Update hooks numérotés	Chaque mise à jour s'exécute une seule fois, dans l'ordre, traçable

14 Commandes Drush utiles

Commande	Rôle
`drush en appointment -y`	Activer le module
`drush updb -y`	Exécuter les mises à jour BDD
`drush cr`	Vider le cache
`drush cron`	Lancer le cron (rappels)
`drush cget appointment.settings`	Voir la configuration
`drush watchdog:show -count=20`	Voir les logs récents
`drush sqlq "SHOW TABLES LIKE 'agency';"`	Vérifier les tables

15 Dépannage

Problème	Solution
500 sur `/appointment/book`	`drush updb -y && drush cr` — tables manquantes
Menu déroulant agences vide	`drush updb -y` — seed data
« Class does not exist »	Vérifier `routing.yml` — simple backslash `\Drupal\...`
Modifications non visibles	Toujours `drush cr` après modif PHP/YAML
Emails non reçus	Vérifier `system.mail.interface.default = SMTPMailSystem`
Rappels non envoyés	`drush cron` + vérifier `reminder_hours_before`

Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement Drupal 10 : entités personnalisées pour la modélisation des données, services pour la logique métier, formulaires multi-étapes avec TempStore, hooks pour étendre le comportement du CMS, système de file d'attente pour les tâches asynchrones, et Batch API pour les opérations lourdes.

Le module `appointment` couvre l'ensemble du cycle de vie d'un rendez-vous — de la réservation à l'annulation — avec une interface d'administration complète et un système de notifications email automatisé. Chaque composant suit les patterns recommandés par Drupal, garantissant un code maintenable, testable et extensible.